

Handwritten Digit Recognition using Machine Learning

Submitted By	Aryan Thakur
Organization	SMARTED INNOVATIONS
Project Type	Individual Internship Project
Submission Date	29 May 2026
Technology Used	Python, Scikit-learn, NumPy, Matplotlib, Seaborn

Abstract

This project implements a handwritten digit recognition system using Logistic Regression and the sklearn digits dataset. The project classifies handwritten digits from 0 to 9 with high accuracy using machine learning techniques.

Introduction

Handwritten digit recognition is an important application of machine learning and computer vision. It is widely used in cheque processing, OCR systems, postal automation, and digital form processing.

Objectives

- Develop a handwritten digit recognition system.
- Train a Logistic Regression classification model.
- Evaluate model performance using accuracy metrics.
- Visualize predictions and confusion matrix outputs.

Methodology

The workflow includes importing libraries, loading the sklearn digits dataset, splitting data into training and testing sets, training the Logistic Regression model, generating predictions, and evaluating performance using classification metrics.

Algorithm Used

Logistic Regression is a supervised machine learning algorithm used for classification tasks. The model learns patterns from handwritten digit images and predicts labels from 0–9.

Results and Observations

The trained model achieved approximately 97% accuracy on the testing dataset. The confusion matrix and classification report demonstrate strong predictive performance across all digit classes.

Conclusion

The project successfully implemented a handwritten digit recognition system using machine learning techniques. The Logistic Regression model accurately classified handwritten digits with strong overall performance.

Future Scope

- Implement CNN-based deep learning models.
- Add GUI support for handwritten input.
- Deploy as a Flask web application.
- Extend the system for alphabet recognition.

References

1. Scikit-learn Documentation
2. Python Official Documentation
3. Matplotlib Documentation
4. Seaborn Documentation

Project Code Structure

Handwritten Digit Recognition using Machine Learning

Import Libraries

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

✓ 0.0s Python

Load & Split Dataset

```
digits = load_digits()
X = digits.data
y = digits.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

✓ 0.0s Python

Create Model

```
model = LogisticRegression(max_iter=10000)
```

✓ 0.0s Python

Train Model

```
model.fit(X_train, y_train)
```

✓ 0.1s Python

LogisticRegression

Predictions & Accuracy

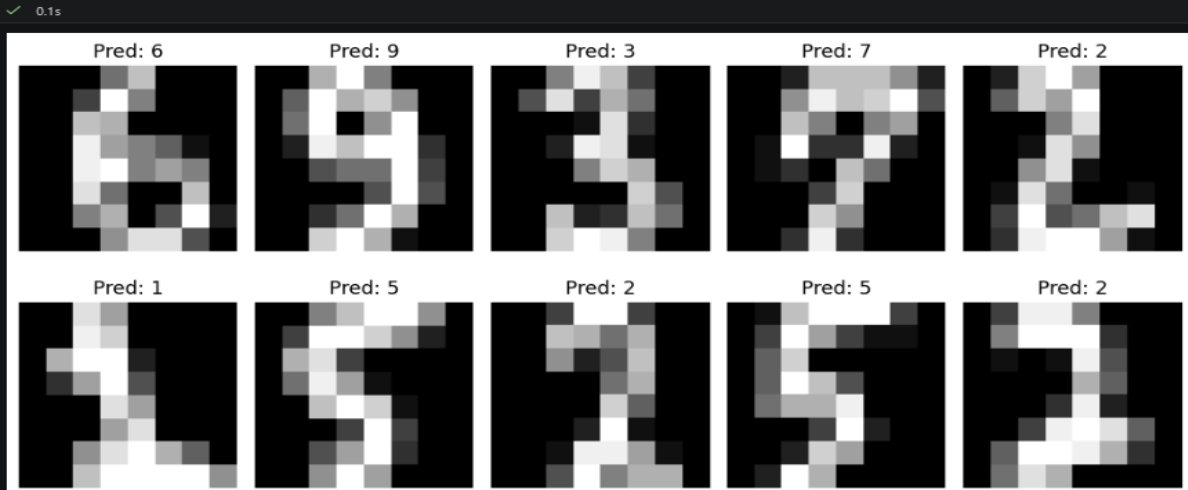
```
y_pred = model.predict(X_test)
```

✓ 0.0s Python

Prediction Output

Sample Prediction

```
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_test[i].reshape(8, 8), cmap='gray')
    plt.title(f"Pred: {y_pred[i]}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```



Confusion Matrix

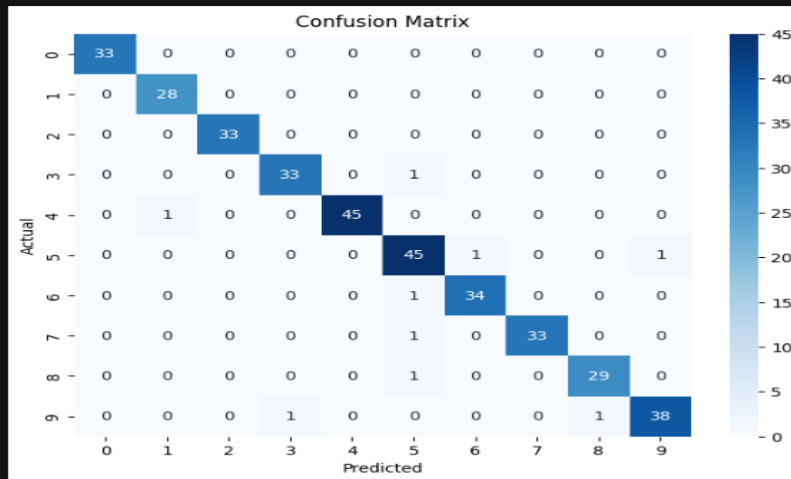
Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```



Classification Report

Classification Report

```
from sklearn.metrics import classification_report  
print(classification_report(y_test, y_pred))
```

21] ✓ 0.0s

```
precision    recall  f1-score   support  
  
0           1.00      1.00      1.00         33  
1           0.97      1.00      0.98         28  
2           1.00      1.00      1.00         33  
3           0.97      0.97      0.97         34  
4           1.00      0.98      0.99         46  
5           0.92      0.96      0.94         47  
6           0.97      0.97      0.97         35  
7           1.00      0.97      0.99         34  
8           0.97      0.97      0.97         30  
9           0.97      0.95      0.96         40  
  
accuracy                0.97        360  
macro avg              0.98        0.98        0.98        360  
weighted avg           0.98        0.97        0.98        360
```